

Arquitetura de Computadores

Eng. Informática, ESTiG/IPB

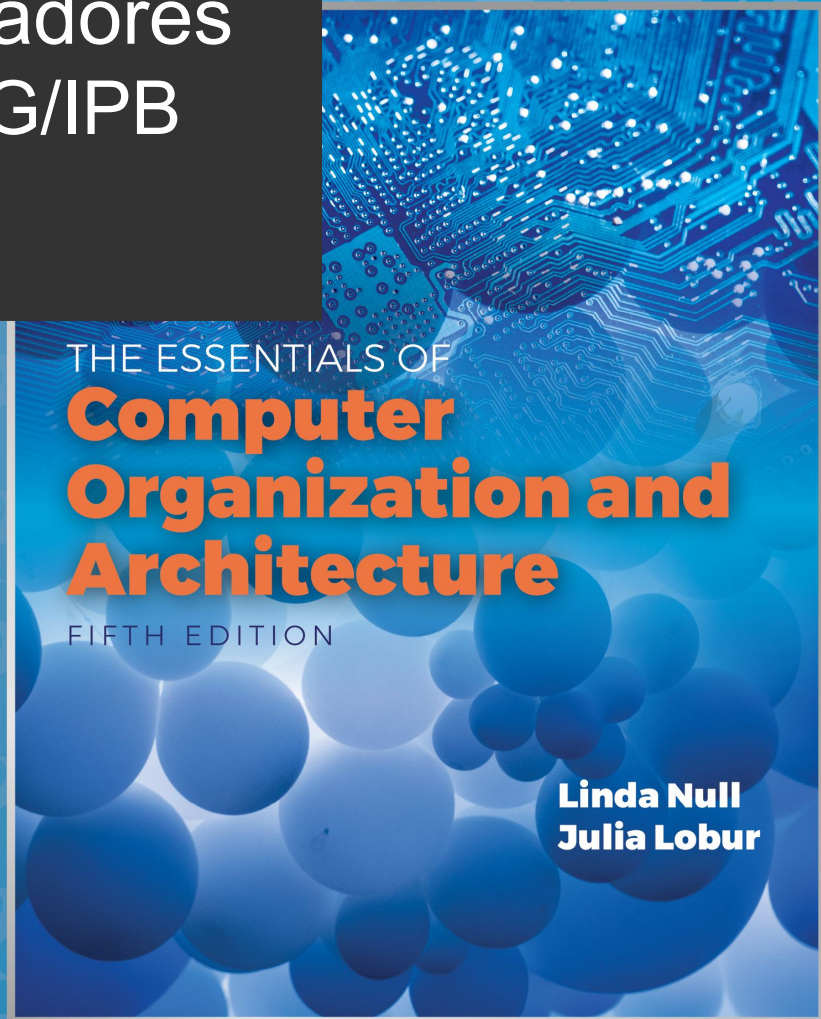
2023/2024

José Rufino

(baseado no Capítulo 4 do livro “The Essentials of Computer Organization and Architecture” de Linda Null e Julia Lobur)

Unidade 3:

Arquitetura MARIE



Objetivos

- conhecer os componentes básicos de uma CPU baseada na arquitetura de von Neumman e a organização básica da Memória Principal (MP)
- ser capaz de explicar o contributo desses componentes, e da Memória Principal, para a execução de programas
- compreender uma arquitetura simples ilustrativa e ser capaz de construir programas para essa arquitetura
- entender o processo de geração de código máquina a partir de *assembly*, e a descodificação das instruções

3.0 Introdução

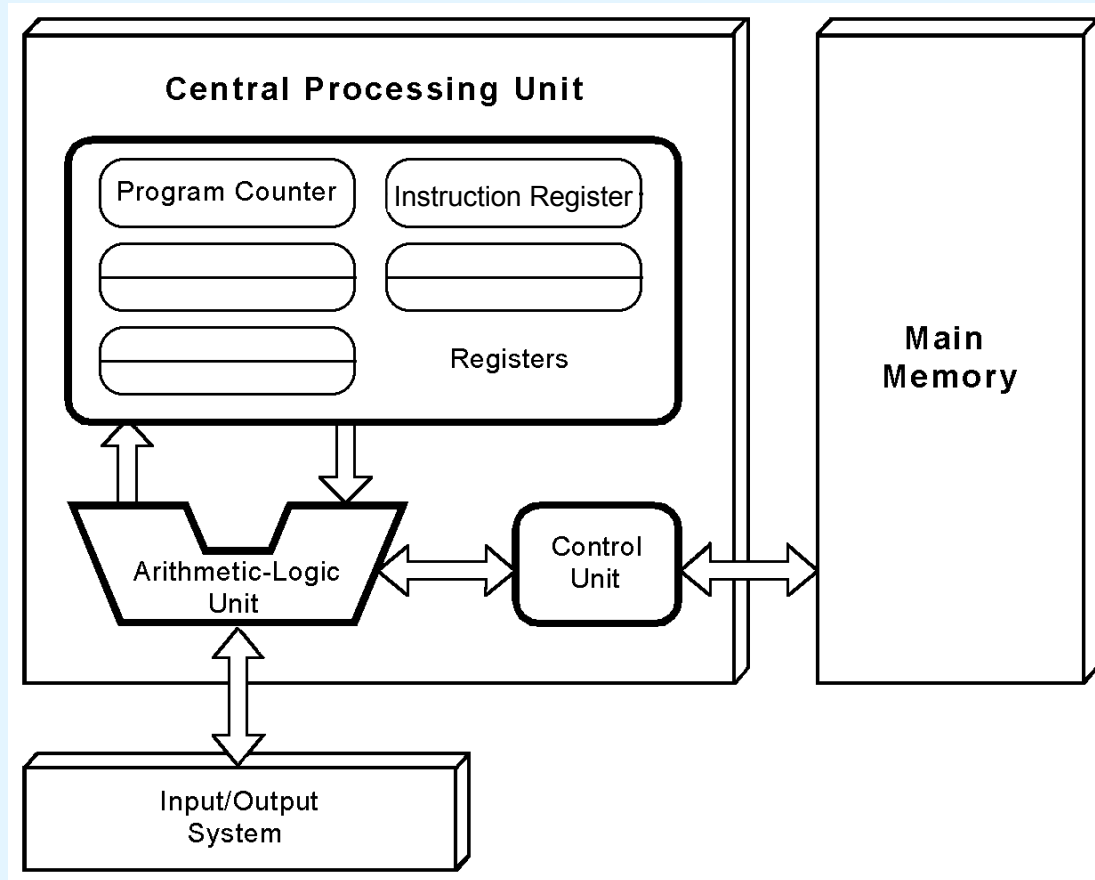


- na unidade 1 foi apresentada uma visão geral dos sistemas de computação
- na unidade 2 apresentou-se a forma como os dados são armazenados e manipulados pelo computador
- na disciplina de Sistemas Digitais descreveram-se componentes fundamentais dos circuitos digitais
- nesta unidade aprofunda-se a compreensão da forma como uma CPU baseada no modelo de von Neumman executa os programas

3.1 Introdução

- Componentes Básicos da CPU

- esquema genérico de um sistema de computação baseado na arquitetura de von Neumann →



3.1 Introdução

- Componentes Básicos da CPU



- **CPU:** lê, descodifica e executa instruções de programas
- as duas principais partes de uma CPU são 1) o **caminho de dados (*datapath*)** e 2) a **unidade de controlo**
 - o *datapath* é constituído por uma unidade lógico-aritmética e por unidades de armazenamento (registos) interligados por um barramento de dados que também liga com a memória principal
 - vários componentes da CPU executam operações sequencialmente, de acordo com as indicações da **unidade de controlo**

3.1 Introdução

- Componentes Básicos da CPU



- **registos** são “unidades de armazenamento” básicas:
 - localizadas na CPU
 - pequena capacidade (8, 16, 32, 64 bits)
 - acesso muito rápido (≤ 1 ciclo de relógio)
 - número variável conforme a arquitetura / CPU
 - tipicamente, uma potência 2
 - com funções variadas
 - guardam qualquer coisa (genéricos), só guardam dados, só guardam endereços, só guardam informação de estado (por exemplo, para assinalar condições de erro da ALU)
 - endereçados de forma diferente da memória principal

3.1 Introdução

- Componentes Básicos da CPU



- **unidade lógico-aritmética (ALU):**
 - executa operações lógicas (e.g., comparações) ...
 - ... e aritméticas (e.g., adições, multiplicações, ...) ...
 - ... às ordens da **unidade de controlo**
 - em geral, tem dois registos de entrada e um de saída
 - as operações afetam um *registo de estado* (e.g., overflow)
- **unidade de controlo:**
 - ***fetch*** da instrução referenciada pelo registo **Program Counter**
 - ***decode*** da instrução, incluindo *fetch* de operandos da memória
 - instrui/programa a ALU para a execução (***execute***) da instrução
 - monitoriza o *registo de estado* e salvaguarda os resultados
 - atende interrupções (notificações assíncronas de eventos)

3.1 Introdução

- Componentes Básicos da CPU



- **relógio da CPU:**

- qualquer computador tem um **relógio de sistema** na placa-mãe, para sincronizar as atividades dos vários componentes
- um certo número de **ciclos de relógio** são necessários para realizar uma movimentação de dados ou uma computação
- a **frequência do relógio** (medida em Hz – ciclos por segundo) define a velocidade com que estas operações são realizadas
- o **tempo de ciclo** (ou **período**) é o recíproco da **frequência do relógio** (um relógio de 800MHz tem um período de 1.25ns)
- na CPU, o sinal de relógio do sistema é multiplicado, gerando um sinal interno: o **relógio [interno] da CPU**, é usado para sincronizar todas as atividades internas realizadas na CPU

3.1 Introdução

- Componentes Básicos da CPU

- **relógio da CPU (cont.):**

- geral/ uma frequência de relógio alta beneficia o desempenho
- mas há outros fatores a considerar - ver a expressão seguinte:

$$\text{CPU Time} = \frac{\text{seconds}}{\text{program}} = \frac{\text{instructions}}{\text{program}} \times \frac{\text{avg. cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}}$$

- o tempo de execução de um programa pode melhor
 - diminuindo o número de instruções do programa, e/ou
 - diminuindo o número de ciclos por cada instrução, e/ou
 - diminuindo o número de segundos por ciclo do relógio da CPU
 - e só este fator é que depende da frequência do relógio !

Esta expressão será revisitada noutros contextos ...

3.1 Introdução

- Organização Básica da Memória Principal

- **memória principal** (também conhecida por **RAM**)
 - é um **veter** (*array*) de **células de armazenamento**, todas do mesmo tamanho (múltiplo de 8 bits)
 - cada célula tem um **endereço único** (inteiro ≥ 0)
 - com **C** células, a **gama de endereços** é $\{0, 1, \dots, C-1\}$
 - tipicamente, C é uma **potência de 2**; donde os números de **0** a **C-1** escrevem-se com **$\log_2 C$ bits**
 - se cada célula da memória armazenar **B** bits, a memória terá uma capacidade total de **$C \times B$ bits** ou **$(C \times B)/8$ bytes**
 - **exemplo:** $C = 4K = 4096 = 2^{12}$: os endereços vão de 0 a 4095 e em binário escrevem-se com 12 bits; $B = 8$ bits: a capacidade total da memória é de 4096×8 bits ou 4096 bytes = 4Kbytes

3.1 Introdução

- MARIE

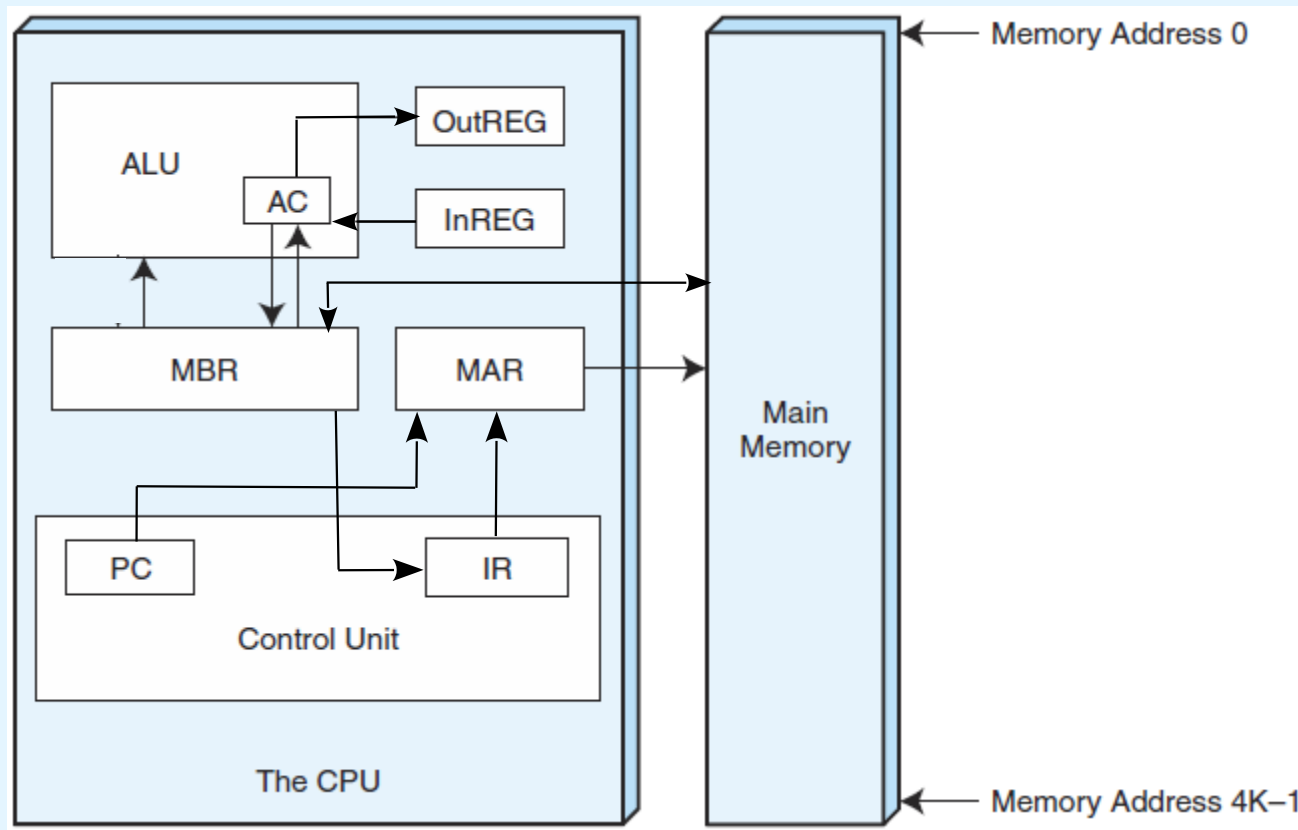


- muitos dos conceitos já introduzidos podem ser melhor compreendidos com um modelo básico de computador
 - o computador MARIE (*Machine Architecture that is Really Intuitive and Easy*) foi desenhado com o propósito de ilustrar conceitos básicos dos sistemas de computação
 - apesar de o MARIE ser demasiado simples para fazer qualquer coisa de útil no mundo real, o estudo das suas funcionalidades permitirá compreender melhor as várias arquiteturas de sistemas de computação reais ...

3.2 MARIE

- Estrutura e Organização da Arquitetura

Representação dos Componentes e Fluxos



Nos slides seguintes, fornecem-se vários detalhes sobre os componentes e, mais tarde, sobre os fluxos gerados pelo ciclo fetch-decode-execute

3.2 MARIE

- Estrutura e Organização da Arquitetura

Memória:

- programas armazenados em memória de 4K palavras
 - endereços de 0 a 4K-1 (endereços de **12** bits)
- tamanho de palavra (largura da memória) = 16 bits
 - palavras de dados com 16 bits
 - números em complemento para 2
 - palavras de instruções com 16 bits
 - 4 bits para opcode, **12** bits para um parâmetro (o parâmetro pode não ser usado, pode ser um endereço de memória, ou uma extensão do opcode)
 - capacidade da memória: 4K x 16 bits = 8 Kbytes

3.2 MARIE

- Estrutura e Organização da Arquitetura

Registos:

- **acumulador (AC)** – registo de **16 bits** que armazena um resultado de uma operação ou um operando de uma instrução com 2 operandos
- **registo de endereço de memória (MAR)** – registo de **12 bits** que armazena um endereço de memória (de uma instrução ou do operando de uma instrução)
- **registo de *buffer* de memória (MBR)** – registo de **16 bits** que armazena os dados antes/depois de uma escrita/leitura para/da memória

3.2 MARIE

- Estrutura e Organização da Arquitetura

Registos (cont.):

- **contador de programa (PC)** – registo de **12 bits** que guarda o endereço da próxima instrução a ser executada
- **registo de instrução (IR)** – registo de **16 bits** que guarda uma instrução imediatamente antes da sua execução
- **registo de entrada de dados (InREG)** – registo de **8 bits** que armazena dados lidos de um dispositivo de entrada
- **registo de saída de dados (OutREG)** – registo de **8 bits** que guarda dados a enviar para um dispositivo de saída

3.3 MARIE

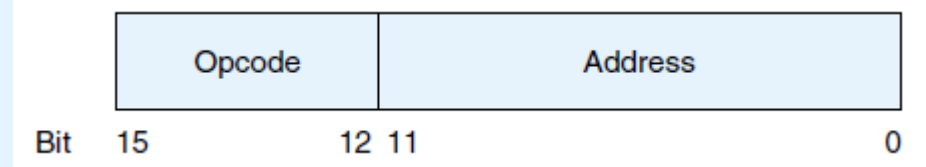
- Arquitetura do Conjunto de Instruções

- a Arquitetura do Conjunto de Instruções (ISA) de um computador especifica o **formato** das suas instruções e as **operações primárias** que a máquina pode fazer
 - a ISA é o interface entre o hardware e o software
 - certas ISAs incluem centenas de instruções diferentes
 - a ISA MARIE inclui apenas 15 instruções:
 - 9 instruções fundamentais (a apresentar já a seguir)
 - 6 instruções complementares (a ver posteriormente)

3.3 MARIE

- Arquitetura do Conjunto de Instruções

- **formato** de uma instrução MARIE:



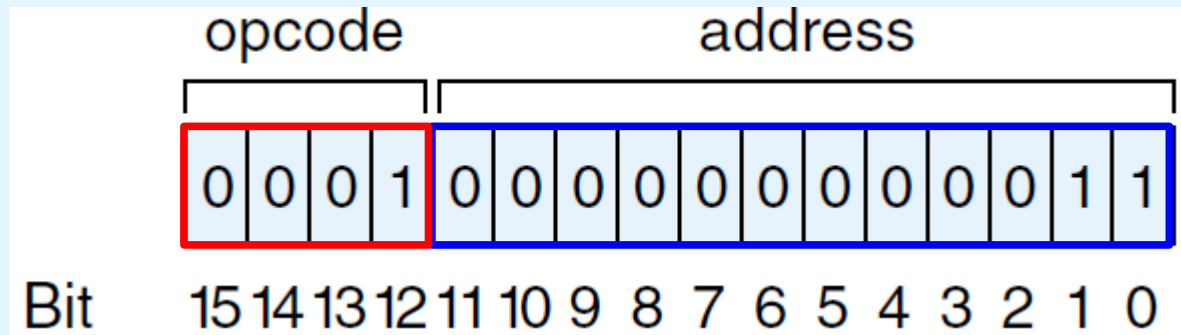
- **instruções** MARIE fundamentais:

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

3.3 MARIE

- Arquitetura do Conjunto de Instruções

- padrão de bits da instrução **LOAD** **003** no registo IR:



- código de operação (*opcode*): $0001(2)=1(16)=1(10)$
- endereço dos dados: $000000000011(2) = 003(16)=3(10)$
 - neste caso, 3 é o endereço final dos dados, pois está-se a usar o **modo de endereçamento direto** (mais adiante serão referidos outros modos)

checkpoint: exercício MD1

3.4 MARIE

- Assemblagem

- as **mnemónicas** *assembly*, tal como **LOAD 104**, são fáceis de escrever e compreender pelos humanos ...
- ... mas não são inteligíveis para o computador
- os **assemblers** traduzem **mnemónicas** (instruções compreensíveis para os humanos) em **linguagem máquina** (compreensível para os computadores)
 - note-se a diferença entre **assembler** e **compilador**:
 - no mundo dos *assemblers* existe uma correspondência de um-para-um entre mnemónicas e código máquina;
 - nos compiladores, normalmente, isso não acontece

3.4 MARIE

- Assemblagem



- um assembler cria, em 2 passos, um ficheiro de código objeto a partir das mnemónicas do código fonte
- **passo 1:** traduzir o máximo de mnemónicas que se conseguir e construir uma ***tabela de símbolos*** que contém referências de memória (endereços) para todos os símbolos que aparecem no programa
- **passo 2:** o código máquina é completado usando os valores (endereços) da tabela de símbolos

3.4 MARIE

- Assemblagem

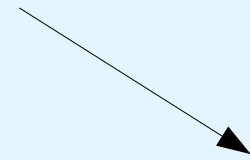
- considere-se o programa apresentado à direita
 - notar o uso de *diretivas* (**HEX** e **DEC**) que especificam a base de representação das 3 constantes
- durante o passo 1, produz-se a tabela de símbolos e o código máquina parcial:



X	104
Y	105
Z	106

// exemplo 3.0

Address	Instruction
100	Load X
101	Add Y
102	Store Z
103	Halt
104 X,	DEC 35
105 Y,	DEC -23
106 Z,	HEX 0000



1	X
3	Y
2	Z
7	0 0 0

3.4 MARIE

- Assemblagem

- após o passo 2, o código máquina fica completo :

1	1	0	4
3	1	0	5
2	1	0	6
7	0	0	0
0	0	2	3
F	F	E	9
0	0	0	0

X	104
Y	105
Z	106

// exemplo 3.0

Address	Instruction
100	Load X
101	Add Y
102	Store Z
103	Halt
104 X,	DEC 35
105 Y,	DEC -23
106 Z,	HEX 0000

checkpoint: exercícios
MAT1.a, MAT2.a,
MAT3.a, MAT4.a,
MAT5.a, MAT6.1

3.5 MARIE

- Linguagem de Transferência de Registos

- cada instrução consiste numa sequência de instruções mais básicas, denominadas de **micro-operações**
- a sequência de micro-operações que realizam uma instrução descreve-se na Linguagem de Transferência de Registos / **Register Transfer Language (RTL)**
- Em MARIE RTL, é usada a seguinte notação:
 - **M[X]** : valor armazenado na posição de memória X
 - **destino ← origem** : transferência de bytes da origem (*) para um destino (*); (*)=registo ou endereço de memória

3.5 MARIE

- Linguagem de Transferência de Registos

Opcode	Instruction	RTN
0001	Load X	$MAR \leftarrow X$ (*) $MBR \leftarrow M[MAR], AC \leftarrow MBR$
0010	Store X	$MAR \leftarrow X$ (*) $MBR \leftarrow AC, M[MAR] \leftarrow MBR$
0011	Add X	$MAR \leftarrow X$ (*) $MBR \leftarrow M[MAR]$ $AC \leftarrow AC + MBR$
0100	Subt X	$MAR \leftarrow X$ (*) $MBR \leftarrow M[MAR]$ $AC \leftarrow AC - MBR$
0101	Input	$AC \leftarrow InREG$
0110	Output	$OutREG \leftarrow AC$
0111	Halt	
1000	Skipcond	If $IR[11-10] = 00$ then If $AC < 0$ then $PC \leftarrow PC + 1$ Else If $IR[11-10] = 01$ then If $AC = 0$ then $PC \leftarrow PC + 1$ Else If $IR[11-10] = 10$ then If $AC > 0$ then $PC \leftarrow PC + 1$
1001	Jump X	$PC \leftarrow IR[11-0]$

(*) durante a
descodificação

3.6 MARIE

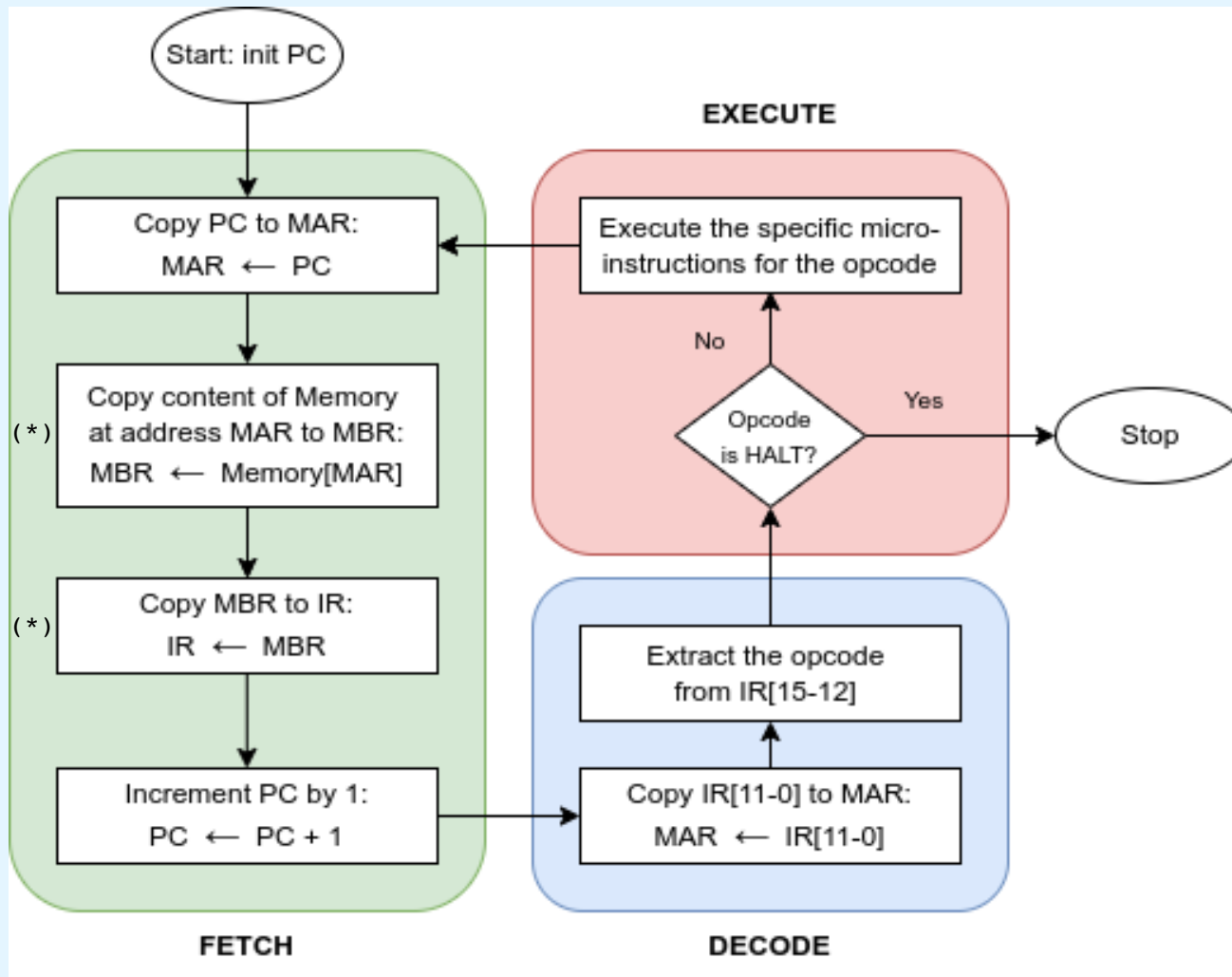
- Ciclo *fetch-decode-execute*



- O ciclo ***fetch-decode-execute*** em MARIE é a série de passos seguidos para executar um programa:
 - **passo 1 (fetch)**: ler instrução da memória para o IR
 - **passo 2 (decode)**: decodificar a instrução para determinar a sua natureza e eventuais parâmetros
 - se for necessário um valor da memória, ler o mesmo
 - **passo 3 (execute)**: executar a instrução
 - pode resultar na escrita de um valor em memória (caso de `Store X`, que guarda AC na posição X)

3.6 MARIE

- Ciclo *fetch-decode-execute*



(*) no livro de apoio, a instrução não passa pelo MBR, indo diretamente para o IR

3.7 MARIE

- Diagramas de Traçagem

- **Diagramas de Traçagem:** mostram a evolução do estado da CPU durante a execução de um programa
- Os slides seguintes mostram estes diagramas para o programa anteriormente assemblado (exemplo 3.0)

Hex Address	Instruction	Binary Contents of Memory Address	Hex Contents of Memory
100	Load 104	0001000100000100	1104
101	Add 105	0011000100000101	3105
102	Store 106	0010000100000110	2106
103	Halt	0111000000000000	7000
104	0023	0000000000100011	0023
105	FFE9	1111111111101001	FFE9
106	0000	0000000000000000	0000

3.7 MARIE

- Diagramas de Traçagem

- **LOAD 104:**

Hex Address	Instruction		Hex Contents of Memory
100	Load	104	1104
101	Add	105	3105
102	Store	106	2106
103	Halt		7000
104	0023		0023
105	FFE9		FFE9
106	0000		0000

Step	RTN	PC	IR	MAR	MBR	AC
(initial values)		100	----	---	----	----
Fetch	MAR $\leftarrow$ PC	100	----	100	----	----
	MBR $\leftarrow$ M[MAR] ^(*)	100	----	100	1104	----
	IR $\leftarrow$ MBR ^(*)	100	1104	100	1104	----
	PC $\leftarrow$ PC + 1	101	1104	100	1104	----
Decode	MAR $\leftarrow$ IR[11-0]	101	1104	104	1104	----
	Decode IR[15-12]	101	1104	104	1104	----
Execute	MBR $\leftarrow$ M[MAR]	101	1104	104	0023	----
	AC $\leftarrow$ MBR	101	1104	104	0023	0023

(*) no livro de apoio, a instrução não passa pelo MBR, indo diretamente para o IR

3.7 MARIE

- Diagramas de Traçagem

- **ADD 105:**

Hex Address	Instruction		Hex Contents of Memory
100	Load	104	1104
101	Add	105	3105
102	Store	106	2106
103	Halt		7000
104	0023		0023
105	FFE9		FFE9
106	0000		0000

Step	RTN	PC	IR	MAR	MBR	AC
(initial values)		101	1104	104	0023	0023
Fetch	MAR $\leftarrow$ PC	101	1104	101	0023	0023
	MBR $\leftarrow$ M[MAR](*)	101	1104	101	3105	0023
	IR $\leftarrow$ MBR (*)	101	3105	101	3105	0023
	PC $\leftarrow$ PC + 1	102	3105	101	3015	0023
Decode	MAR $\leftarrow$ IR[11-0]	102	3105	105	3105	0023
	Decode IR[15-12]	102	3105	105	3105	0023
Execute	MBR $\leftarrow$ M[MAR]	102	3105	105	FFE9	0023
	AC $\leftarrow$ AC + MBR	102	3105	105	FFE9	000C

(*) no livro de apoio, a instrução não passa pelo MBR, indo diretamente para o IR

3.7 MARIE

- Diagramas de Traçagem

- STORE 106:**

Hex Address	Instruction		Hex Contents of Memory
100	Load	104	1104
101	Add	105	3105
102	Store	106	2106
103	Halt		7000
104	0023		0023
105	FFE9		FFE9
106	0000		0000

Step	RTN	PC	IR	MAR	MBR	AC
(initial values)		102	3105	105	FFE9	000C
Fetch	MAR $\leftarrow$ PC	102	3105	102	FFE9	000C
	MBR $\leftarrow$ M[MAR] ^(*)	102	3105	102	2106	000C
	IR $\leftarrow$ MBR ^(*)	102	2106	102	2106	000C
	PC $\leftarrow$ PC + 1	103	2106	102	2106	000C
Decode	MAR $\leftarrow$ IR[11-0]	103	2106	106	2106	000C
	Decode IR[15-12]	103	2106	106	2106	000C
Execute	MBR $\leftarrow$ AC	103	2106	106	000C	000C
	M[MAR] $\leftarrow$ MBR	103	2106	106	000C	000C

(*) no livro de apoio, a instrução não passa pelo MBR, indo diretamente para o IR

3.7 MARIE

- Diagramas de Traçagem

- HALT:**

Hex Address	Instruction		Hex Contents of Memory
100	Load	104	1104
101	Add	105	3105
102	Store	106	2106
103	Halt		7000
104	0023		0023
105	FFE9		FFE9
106	0000		0000

Step	RTN	PC	IR	MAR	MBR	AC
(initial values)		103	2106	106	000C	000C
Fetch	MAR $\leftarrow$ PC	103	2106	103	000C	000C
	MBR $\leftarrow$ M[MAR](*)	103	2106	103	7000	000C
	IR $\leftarrow$ MBR (*)	103	7000	103	7000	000C
	PC $\leftarrow$ PC + 1	104	7000	103	7000	000C
Decode	MAR $\leftarrow$ IR[11-0]	104	7000	000	7000	000C
	Decode IR[15-12]	104	7000	000	7000	000C
Execute	Stop	104	7000	000	7000	000C

(*) no livro de apoio, a instrução não passa pelo MBR, indo diretamente para o IR

3.7 MARIE

- Diagramas de Traçagem

Hex Address	Instruction	Hex Contents of Memory
100	Load 104	1104
101	Add 105	3105
102	Store 106	2106
103	Halt	7000
104	0023	0023
105	FFE9	FFE9
106	0000	0000

Traçagem (visão alternativa):

	Load 104	Add 105	Store 106	Halt
MAR:	-,100,104	104,101,105	105,102,106	106,103,000
MBR:	-,1104,0023	0023,3105,FFE9	FFE9,2106,000C	000C,7000
PC:	100,101	101,102	102,103	103,104
IR:	-,1104	1103,3105	3105,2106	2106,7000
AC:	-,0023	0023,000C	000C=000C	000C=000C
vermelho : valor inicial azul : valor final				

Traçagem (visão alternativa compacta):

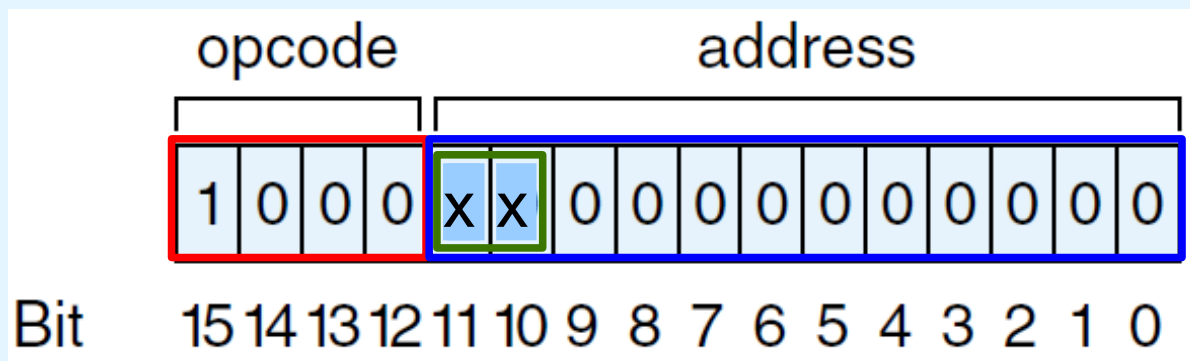
	Load 104	Add 105	Store 106	Halt
MAR:	100,104	101,105	102,106	103,000
MBR:	1104,0023	3105,FFE9	2106,000C	7000
PC:	100,101	102	103	104
IR:	1104	3105	2106	7000
AC:	0023	000C	000C	000C
azul : valor final				

checkpoint: testar simulador (<https://marie.v2.pt>) com exemplo 3.0; exercícios MAT1.b, MAT2.b, MAT3.b, MAT4.b, MAT5.b, MAT6.2

3.8 MARIE

- Saltos Condicionais (If-Then-Else)

- em MARIE, os saltos condicionais do tipo If-Then-Else são implementados pela instrução SKIPCOND
- padrão de bits da instrução **SKIPCOND** no registo IR:



- código de operação (*opcode*): $1000(2)=8(16)=8(10)$
- se os bits nas posições 11 e 10 tomarem os valores “00”, “01”, “10”, então a próxima instrução não será executada (PC++) se o valor de AC for $<$, $=$ ou > 0

3.8 MARIE

- Saltos Condicionais (If-Then-Else)

```
// exemplo 3.1
int x=12, y=20;
if (x==y) x = 2 * x;
else y = y - x;
printf("%d", y);
```

checkpoint:
exercícios MC1, MC2,
MC3, MC4, MC5

If,	Load	X	/Load the first value
	Subt	Y	/Subtract value of Y and store result in AC
	Skipcond	400	/If AC = 0, skip the next instruction
	Jump	Else	/Jump to the Else part if AC is not equal to 0
Then,	Load	X	/Reload X so it can be doubled
	Add	X	/Double X
	Store	X	/Store the new value
	Jump	Endif	/Skip over Else part to end of If
Else,	Load	Y	/Start the Else part by loading Y
	Subt	X	/Subtract X from Y
	Store	Y	/Store Y - X in Y
Endif,	Load	Y	/Reload Y so it can be printed
	Output		/Print Y
	Halt		/Terminate program
X,	Dec	12	/12 is the initial value of X
Y,	Dec	20	/20 is the initial value of Y

3.9 MARIE

- Instruções Complementares

- as instruções MARIE apresentadas até agora (as fundamentais) usam *endereçamento direto*:
 - a instrução inclui o endereço do operando
- outras instruções usam *endereçamento indireto*:
 - a instrução inclui o endereço de uma célula de memória (*), a qual guarda o endereço do operando pretendido

Instruction Number (hex)	Instruction	Meaning
0	JnS X	Store the PC at address X and jump to X + 1.
A	Clear	Put all zeros in AC.
B	AddI X	Add indirect: Go to address X. Use the value at X as the actual address of the data operand to add to AC.
C	JumpI X	Jump indirect: Go to address X. Use the value at X as the actual address of the location to jump to.
D	LoadI X	Load indirect: Go to address X. Use the value at X as the actual address of the operand to load into the AC.
E	StoreI X	Store indirect: Go to address X. Use the value at X as the destination address for storing the value in AC.

(*) ou seja, esta célula é um apontador !

3.9 MARIE: Instruções Complementares

- Descrição RTL

JnS

MAR $\leftarrow$ X (*)

MBR $\leftarrow$ PC

M[MAR] $\leftarrow$ MBR

MBR $\leftarrow$ X

AC $\leftarrow$ 1

AC $\leftarrow$ AC + MBR

PC $\leftarrow$ AC

Clear

AC $\leftarrow$ 0

Addl X

MAR $\leftarrow$ X (*)

MBR $\leftarrow$ M[MAR]

MAR $\leftarrow$ MBR

MBR $\leftarrow$ M[MAR]

AC $\leftarrow$ AC + MBR

Jumpl X

MAR $\leftarrow$ X (*)

MBR $\leftarrow$ M[MAR]

PC $\leftarrow$ MBR

Loadl X

MAR $\leftarrow$ X (*)

MBR $\leftarrow$ M[MAR]

MAR $\leftarrow$ MBR

MBR $\leftarrow$ M[MAR]

AC $\leftarrow$ MBR

Storel X

MAR $\leftarrow$ X (*)

MBR $\leftarrow$ M[MAR]

MAR $\leftarrow$ MBR

MBR $\leftarrow$ AC

M[MAR] $\leftarrow$ MBR

(*) feito na decodificação

3.9 MARIE: Instruções Complementares

- Loadl X

- **LOADI X** (leitura indireta):
 - “o valor a ler para o registo AC é o apontado por X”
 - X contém o endereço de onde ler o valor a dar a AC
 - **LOADI 400** : se a palavra 400 contiver o valor 240, será na palavra 240 que está o valor a ler para AC
 - descrição RTL:

MAR $\leftarrow$ **X**

MBR $\leftarrow$ **M[MAR]**

MAR $\leftarrow$ **MBR**

MBR $\leftarrow$ **M[MAR]**

AC $\leftarrow$ **MBR**

3.9 MARIE: Instruções Complementares

- Storel X

- **STOREI X** (escrita indireta):
 - “o valor de AC é escrito na posição apontada por X”
 - X contém o endereço onde guardar o valor de AC
 - STOREI 400 : se a palavra 400 contiver o valor 240, será na palavra 240 que será guardado o valor de AC
 - descrição RTL:

MAR $\leftarrow$ **X**

MBR $\leftarrow$ **M[MAR]**

MAR $\leftarrow$ **MBR**

MBR $\leftarrow$ **AC**

M[MAR] $\leftarrow$ **MBR**

3.9 MARIE: Instruções Complementares

- Addl X

- **ADDI X** (adição indireta):
 - “o valor a somar a AC é o apontado por X”
 - X contém o endereço do valor a somar a AC
 - ADDI 400 : se a palavra 400 guardar o valor 240, será na palavra 240 que está o valor a somar a AC
 - descrição RTL
(similar à de **LOADI X**):

```
MAR ← X
MBR ← M[MAR]
MAR ← MBR
MBR ← M[MAR]
AC ← AC + MBR
```

3.9 MARIE: Instruções Complementares

- LoadI, AddI (exemplo)

```
// exemplo 3.2
char a[13]="Hello World!";
int i=0;
while (a[i]!='\0') {
    printf("%c", a[i]);
    i = i + 1;
}
```

```
ORG 100

Loop,    Load    ArrayBaseAddress
        Add      ArrayCellIndex
        Store    ArrayCellAddress / new address of a[i]

        LoadI   ArrayCellAddress
        Skipcond 400 / check if a[i] == '\0'
        Jump    BodyLoop
        Jump    EndLoop

BodyLoop, Output / output a[i]

        Load    ArrayCellIndex
        Add      One
        Store    ArrayCellIndex / i++

        Jump    Loop

EndLoop, Halt
```

```
/ alternative
Clear
AddI ArrayCellAddress
```

Array,	Dec	072	/ H
	Dec	101	/ e
	Dec	108	/ l
	Dec	108	/ l
	Dec	111	/ o
	Dec	032	/ [space]
	Dec	119	/ w
	Dec	111	/ o
	Dec	114	/ r
	Dec	108	/ l
	Dec	100	/ d
	Dec	033	/ !
	Dec	000	/ [null]
ArrayBaseAddress, Hex 10D / address of a[0]			
/ArrayBaseAddress, Hex 10E / alternative			
ArrayCellAddress, Hex 000 / address of a[i]			
ArrayCellIndex,	Dec	0	/ index i
One,	Dec	1	/ constant 1

3.9 MARIE: Instruções Complementares

- LoadI, AddI (exemplo)

```
// exemplo 3.3
int a[5]={10,15,20,25,30};
int sum=0, i=0;
while (i<5) {
    sum = sum + a[i];
    i = i + 1;
}
printf("%d", sum);
```

```

ORG 100

Loop,   Load      ArrayCellIndex
        Subt       Five
        Skipcond   000           / check if i < 5
        Jump      EndLoop

        Load      ArrayBaseAddress
        Add        ArrayCellIndex
        Store      ArrayCellAddress / new address of a[i]

        Load      Sum
        AddI       ArrayCellAddress / sum = sum + a[i]
        Store      Sum

        Load      ArrayCellIndex
        Add        One
        Store      ArrayCellIndex / i++

        Jump      Loop

EndLoop, Load      Sum
        Output
        Halt
    
```

```

/ alternativa
/ sum = a[i] + sum
LoadI ArrayCellAddress
Add    Sum
Store  Sum
    
```

```

Dec 10 / a[0]
Dec 15 / a[1]
Dec 20 / a[2]
Dec 25 / a[3]
Dec 30 / a[4]
ArrayBaseAddress, Hex 111 / address of a[0]
ArrayCellAddress, Hex 000 / address of a[i]
ArrayCellIndex,   Dec 0   / index i
Sum,              Dec 0   / sum
One,              Dec 1   / constant 1
Five,            Dec 5   / constant 5
    
```

3.9 MARIE: Instruções Complementares

- Storel (exemplo)

```
// exemplo 3.4
int a[5], i;
for (i=0; i<5; i++) {
    scanf("%d", &a[i]);
}
```

ORG 100

```
Loop,    Load      ArrayCellIndex
          Subt       Five
          Skipcond   000          / check if i < 5
          Jump      EndLoop

          Load      ArrayBaseAddress
          Add        ArrayCellIndex
          Store      ArrayCellAddress / new address of a[i]

          Input
          StoreI     ArrayCellAddress / read a[i] from user

          Load      ArrayCellIndex
          Add        One
          Store      ArrayCellIndex / i++

          Jump      Loop

EndLoop, Halt
```

checkpoint:
exercício MC6

```
Dec 0 / a[0]
Dec 0 / a[1]
Dec 0 / a[2]
Dec 0 / a[3]
Dec 0 / a[4]
ArrayBaseAddress, Hex 10E / address of a[0]
ArrayCellAddress, Hex 000 / address of a[i]
ArrayCellIndex,   Dec 0   / index i
One,              Dec 1   / constant 1
Five,             Dec 5   / constant 5
```

3.9 MARIE: Instruções Complementares

- Rotinas: JnS X

- **JNS X** (*jump-and-store*): conjugado com a utilização de **JUMPI X**, suporta a execução de **subrotinas**
 - guardar o PC no endereço **X** e saltar para **X+1**, onde começa a subrotina; a subrotina retorna com **JUMPI X**
 - descrição RTL:

MBR $\leftarrow$ **PC**

MAR $\leftarrow$ **X**

M[MAR] $\leftarrow$ **MBR**

MBR $\leftarrow$ **X**

AC $\leftarrow$ 1

AC $\leftarrow$ **AC** + **MBR**

PC $\leftarrow$ **AC**

3.9 MARIE: Instruções Complementares

- Rotinas: JumpI X

- **JUMPI X** (salto indireto) :
 - “o valor a atribuir ao PC é o guardado em X”
 - X contém o endereço da próxima instrução a executar
 - JUMPI 400 : se a palavra 400 guardar o valor 240, será na palavra 240 que está a próxima instrução
 - código RTL:

MAR $\leftarrow$ **X**

MBR $\leftarrow$ **M[MAR]**

PC $\leftarrow$ **MBR**

3.9 MARIE: Instruções Complementares

- Rotinas (exemplo)

```
// exemplo 3.5
int twice (int num) { int res;
    res = num + num; return(res); }
main() {
    int x = 20, y = 48;
    x = twice(x); printf("%d", x);
    y = twice(y); printf("%d", y);
}
```

ORG 100

```
Load X      / load the 1st number (X) to be doubled
Store Num   / use Num as a parameter to pass X to Twice
JnS Twice   / store return address and jump to Twice
Load Res    / load the double of X returned by Twice
Store X     / update X with its double
Output      / show the new value of X
```

```
Load Y      / load the 2nd number (Y) to be doubled
Store Num   / use Num as a parameter to pass Y to Twice
JnS Twice   / store return address and jump to Twice
Load Res    / load the double of Y returned by Twice
Store Y     / update Y with its double
Output      / show the new value of Y
```

```
Halt        / end program
```

```
Twice, Hex 0 / place to store return address
```

```
Load Num    / AC = Num
Add Num     / AC = AC + Num = Num + Num = 2 * Num
Store Res   / Res = 2 * Num
JumpI Twice / return to calling code
```

checkpoint:
exercícios MC7,
MC8, MC11.a,
MC9, MC10,
MC11.c, MC12.c,
MC11.e, MC11.f,
MC12.d, MC13.a

X,	Dec	20
Y,	Dec	48
Num,	Dec	0
Res,	Dec	0

3.10 Descodificação de Instruções

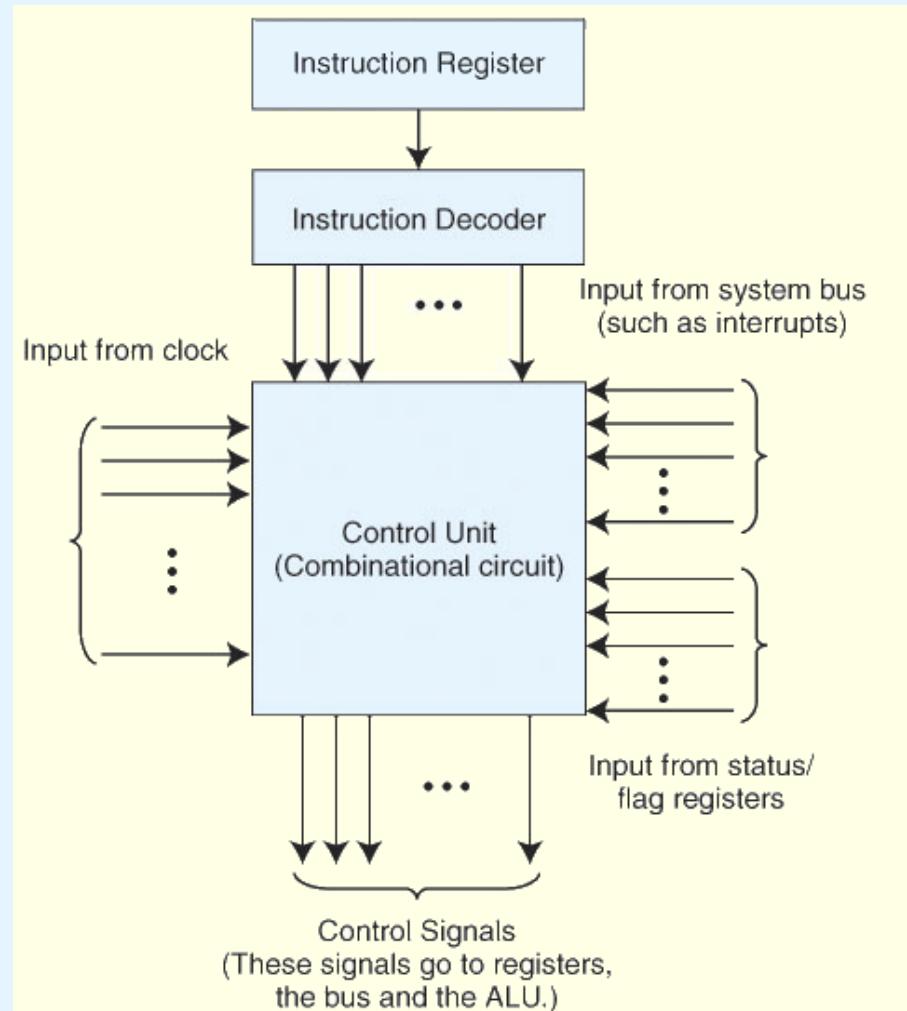


- a unidade de controlo mantém o sistema sincronizado, garantindo que os bits (dados e controlo) chegam ao componente certo qd é necessária a sua intervenção
- formas de implementar uma unidade de controlo (UC):
 - *controlo **hardwired***: a UC é um circuito combinatório e, para uma certa combinação de bits à entrada, ativa um conjunto de linhas à saída, que **controlam** outros *componentes digitais*
 - *controlo **microprogramado***: a UC dispõe de um interpretador (software) que converte cada instrução máquina num conjunto de micro-instruções, e estas em sinais elétricos de **controlo**

3.10 Descodificação de Instruções

controle hardwired:

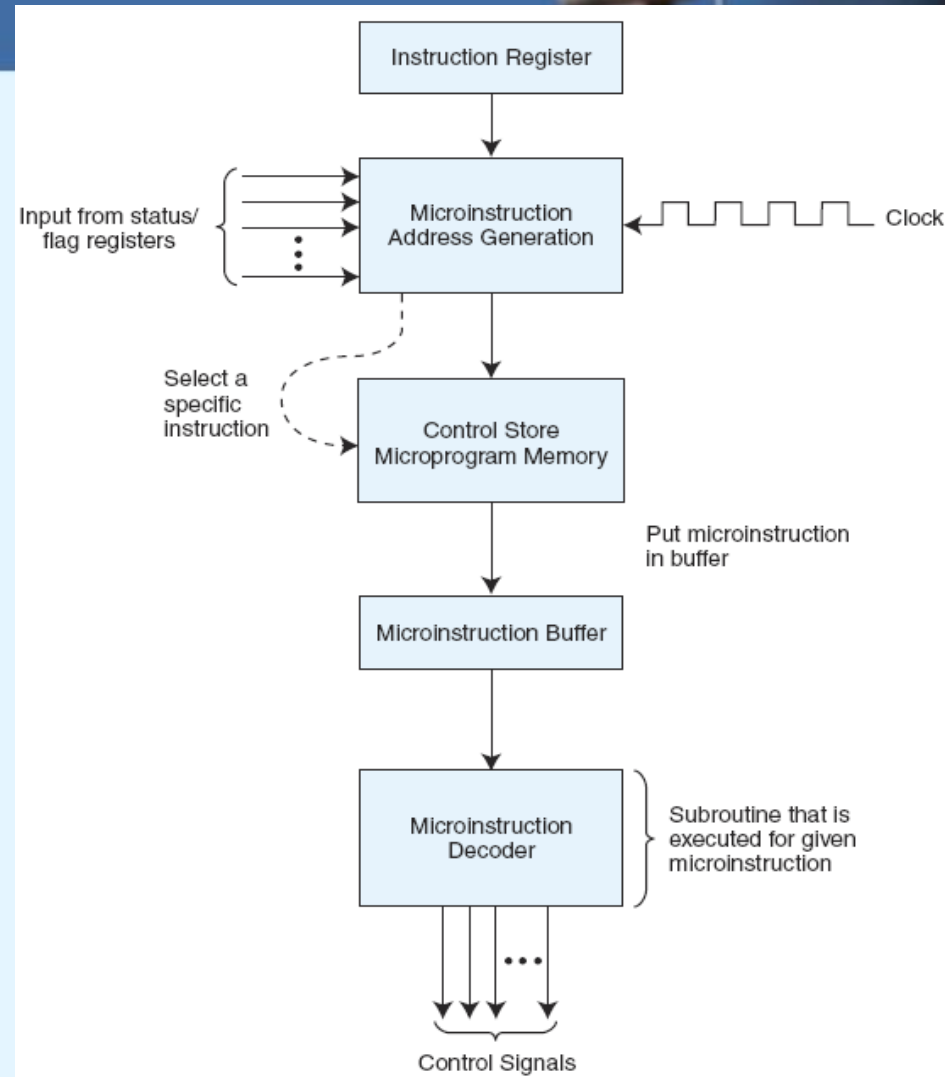
- para decodificar *opcodes* MARIE seria necessário um decodificador 4-para-16
- o diagrama de blocos da direita mostra a configuração genérica de uma unidade de controlo implementada por hardware específico
- (+) : bom desempenho
- (-) : complexo, inflexível



3.10 Descodificação de Instruções

controlo microprogramado:

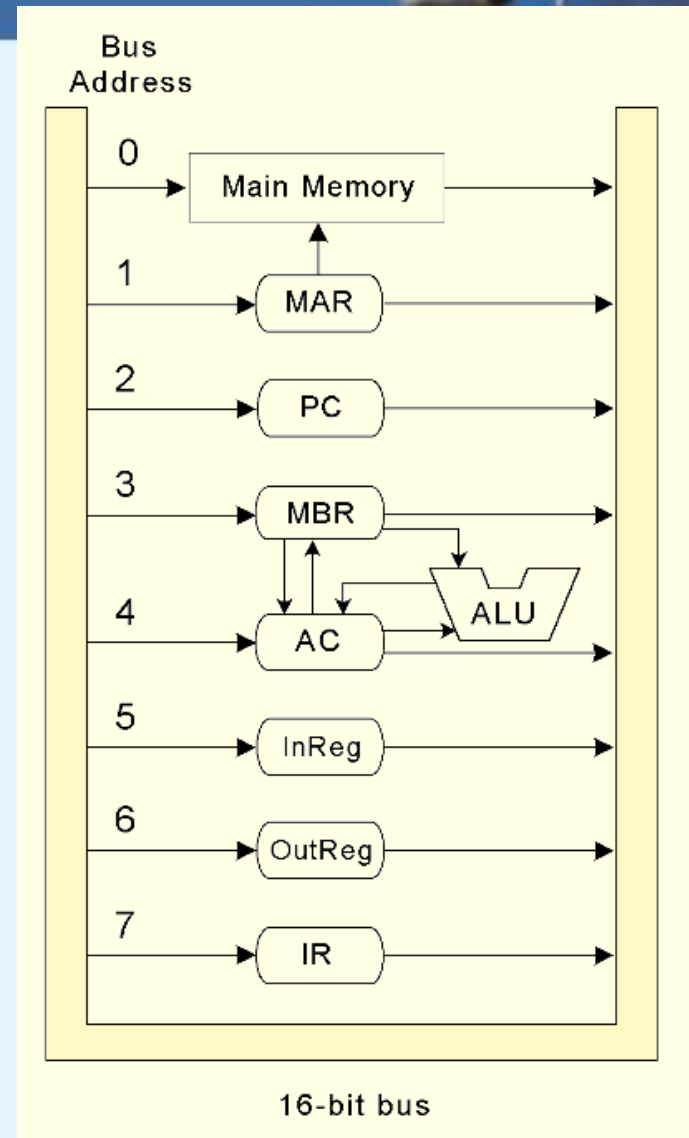
- o interpretador (microprograma) é armazenado como *firmware* numa memória ROM, PROM ou EPROM, designada *control store*
- cada instrução máquina origina várias micro-instruções; existe uma subrotina para cada micro-instrução, que ativará os componentes digitais próprios
- (-) : < desempenho (software)
- (+) : simples, flexível (estender o *instruction-set* corresponde a codificar novas subrotinas)



3.10.1 Descodificação de Instruções em MARIE

- Registos e Memória têm um endereço único (Bus Address, na figura) no Datapath do MARIE →
 - os endereços tomam a forma de combinações de sinais, gerados pela unidade de controlo

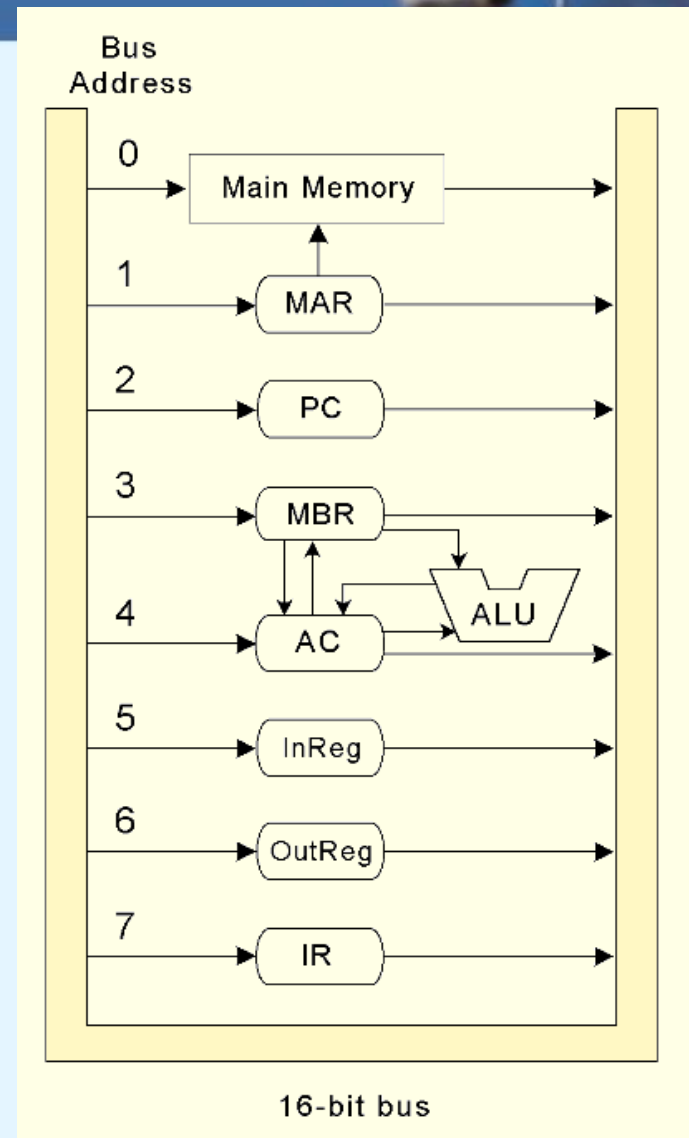
Quantas linhas de sinal necessita a UC do MARIE ?



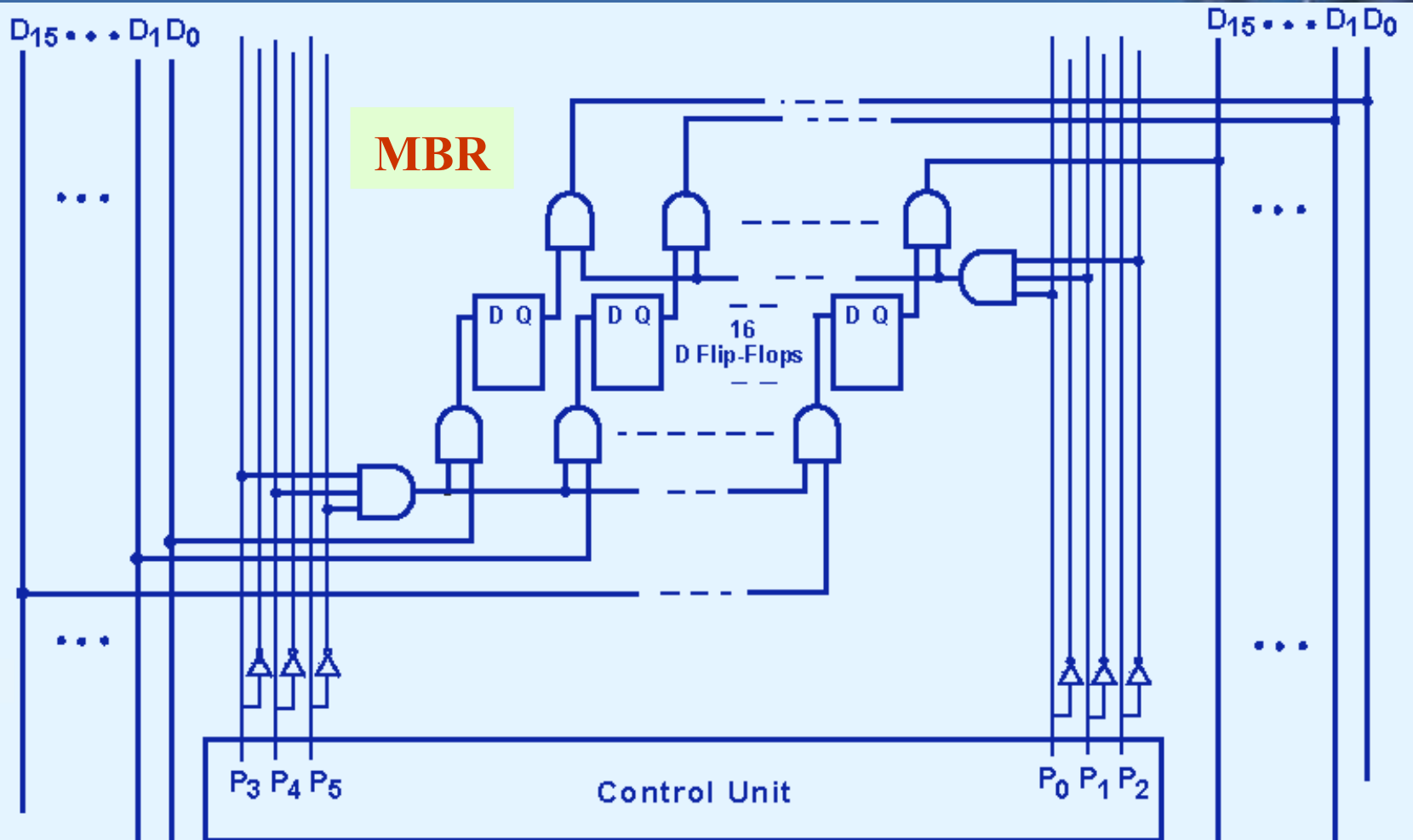
3.10.1 Descodificação de Instruções em MARIE

- 2 conjuntos de 3 sinais:
 - $\{P_0, P_1, P_2\}$ controla a leitura da memória ou de um registo;
 - $\{P_3, P_4, P_5\}$ controla a escrita na memória ou num registo.

O próximo slide mostra o registo MBR do MARIE em detalhe.



3.10.1 Descodificação de Instruções em MARIE



O MBR será lido (escrito) quando P_0 e P_1 (P_3 e P_4) estiverem ativos (note-se que o Bus Address do MBR no Datapath é $3 = 011$)

3.10.1 Descodificação de Instruções em MARIE

- A análise do RTL do MARIE revela que a ALU suporta apenas 3 operações: add, subtract, clear.
 - Podendo-se ainda definir uma quarta: “do nothing”
- Ao todo, o MARIE necessitaria de sinais de controlo:
 - para Registos: P_0 a P_5 .
 - para a ALU: A_0 a A_3
 - para Temporização: T_0 a T_7 , e counter reset C_r

3.10.1 Descodificação de Instruções em MARIE

- Considere-se a instrução Add X, cuja tradução em RTL é:
 $\text{MAR} \leftarrow X$
 $\text{MBR} \leftarrow \text{M}[\text{MAR}]$
 $\text{AC} \leftarrow \text{AC} + \text{MBR}$
- Atente-se no primeiro passo ($\text{MAR} \leftarrow X$):
 - após o fetch, o endereço X está nos 12 bits menos significativos do IR, com endereço 7 no Datapath
 - X é copiado para o MAR, com endereço 1 no Datapath
 - donde, é necessário ativar os sinais P_2 , P_1 , e P_0 ($111 = 7$) para ler do IR e o sinal P_3 ($001=1$) para escrever no MAR

3.10.1 Descodificação de Instruções em MARIE

- A sequência completa de sinais para o Add seria:

$P_0 P_1 P_2 P_3 T_0: \text{MAR} \leftarrow X$

$P_0 P_2 T_1: \text{MBR} \leftarrow M[\text{MAR}]$

$A_0 P_0 P_1 P_2 P_5 T_2: \text{AC} \leftarrow \text{AC} + \text{MBR}$

$C_r T_3: [\text{Reset counter}]$

- Estes sinais são combinados (AND) com circuitos combinatórios para provocar o efeito pretendido.

O próximo slide mostra o diagrama temporal do Add

3.10.1 Descodificação de Instruções em MARIE

- Note-se a evolução dos estados dos vários sinais em simultâneo, em cada ciclo máquina C_0 a C_3 .

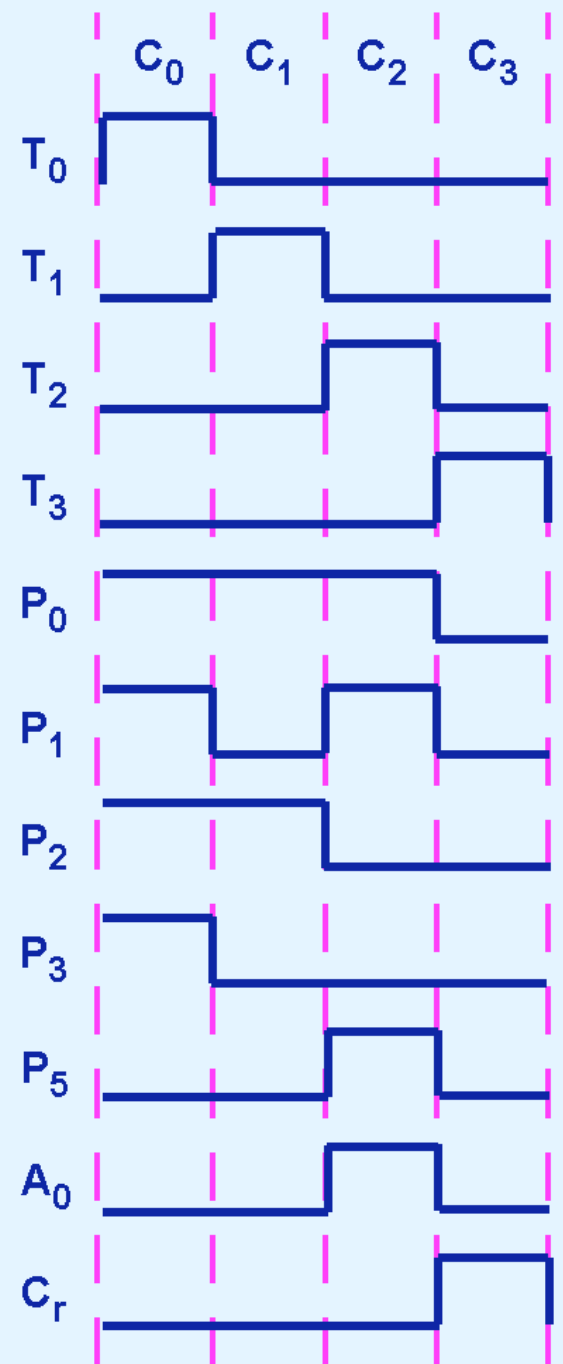
$P_0 P_1 P_2 P_3 T_0$: $MAR \leftarrow X$

$P_0 P_2 T_1$: $MBR \leftarrow M[MAR]$

$A_0 P_0 P_1 P_2 P_5 T_2$: $AC \leftarrow AC + MBR$

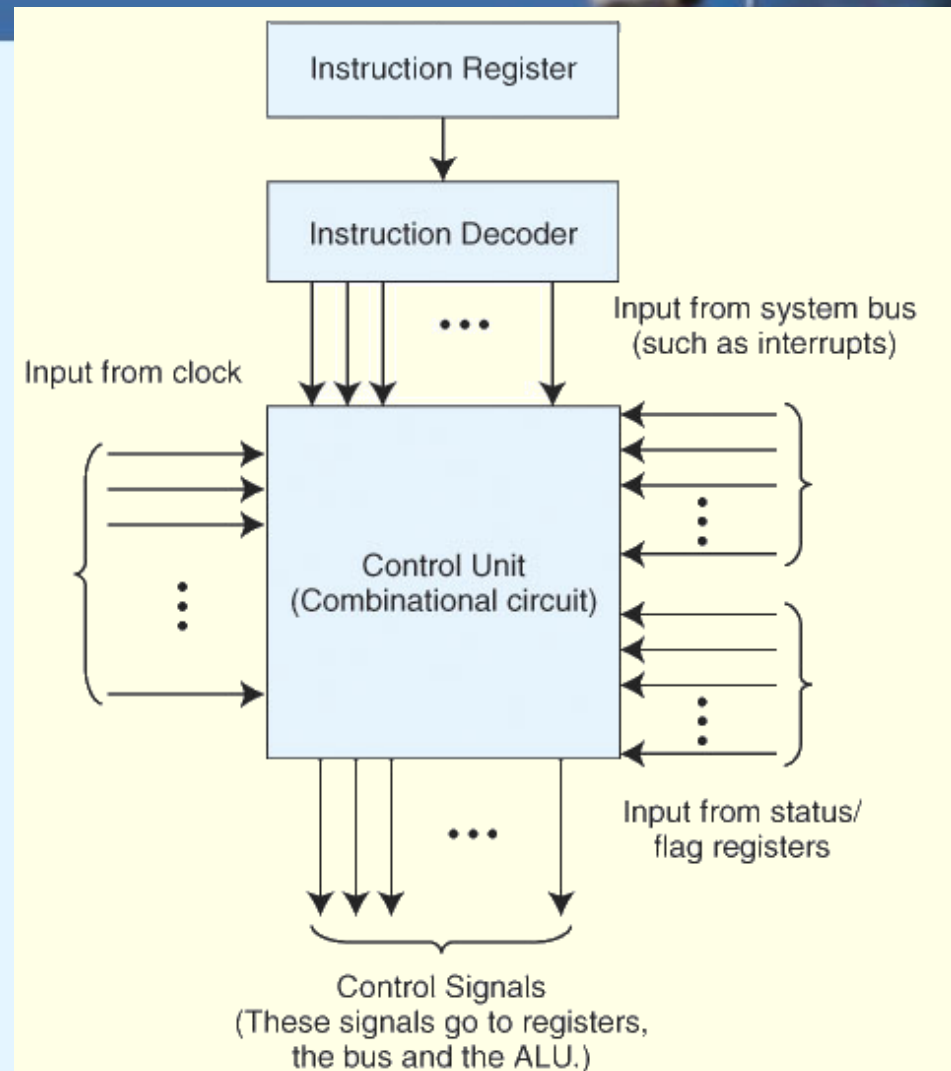
$C_r T_3$: [Reset counter]

No MARIE, a instrução Add necessita de 4 ciclos-máquina

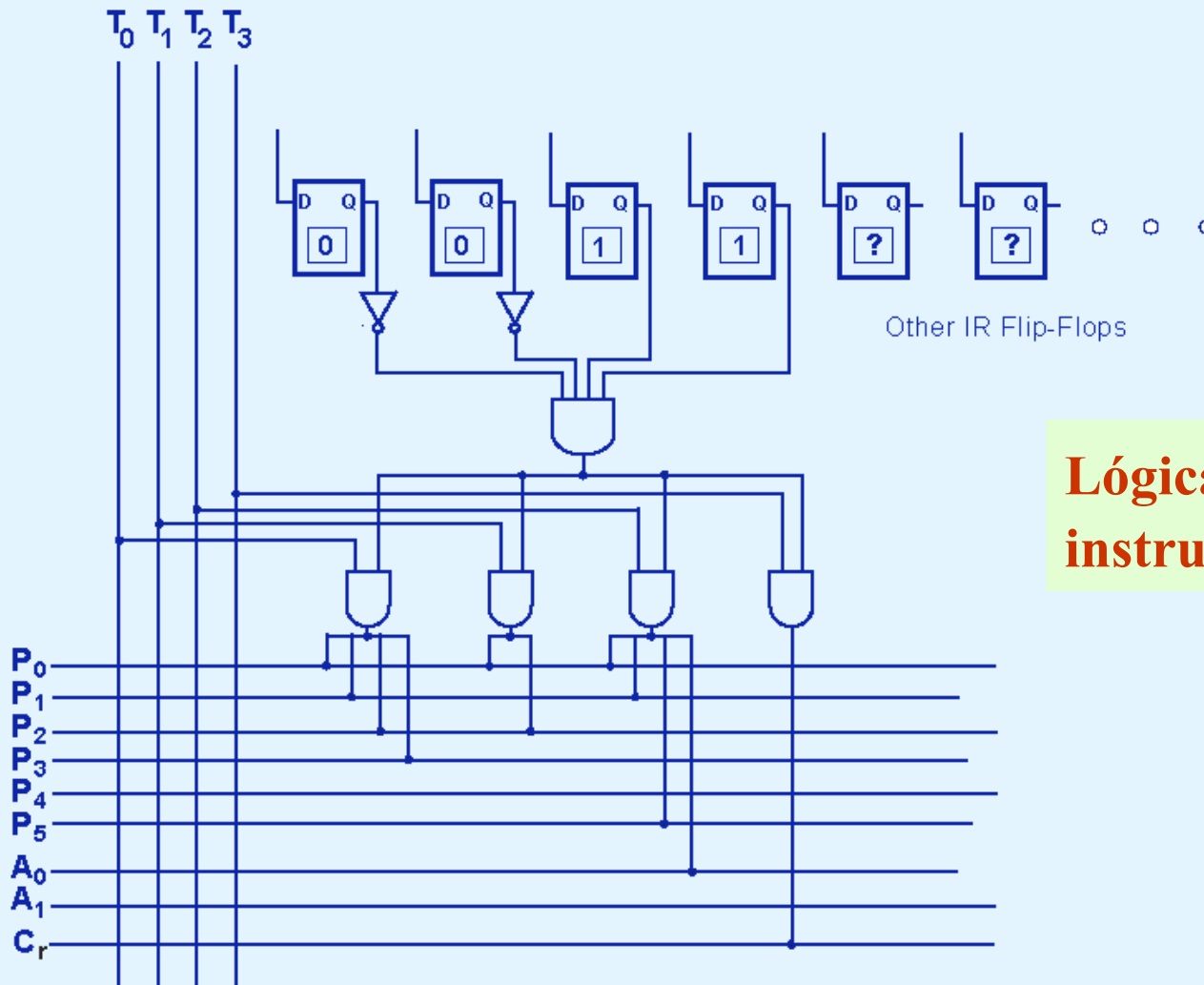


3.10.1 Descodificação de Instruções em MARIE

- O padrão de sinais descrito anteriormente é o mesmo quer se use controlo hardwired, quer microprogramado ...
- No **controlo hardwired**, o padrão de bits de uma instrução “alimenta” diretamente a lógica combinatória da UC.



3.10.1 Descodificação de Instruções em MARIE

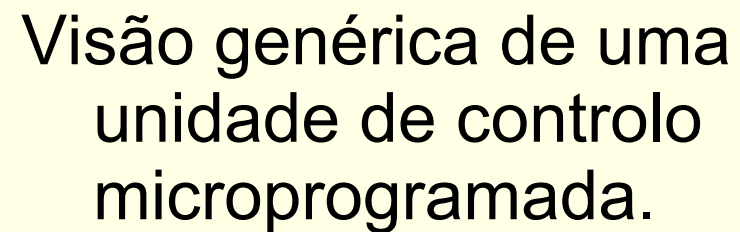


**Lógica hardwired para a
instrução Add = 0011**

3.10.1 Descodificação de Instruções em MARIE

- No **controlo microprogramado**, é o microcódigo que irá provocar mudanças nos sinais de controlo
- As instruções-máquina entram num microprograma que converte 0s e 1s em sinais de controlo.
- O microprograma está gravado sob uma forma de *firmware*, designada de *control store*.
- Em cada ciclo-máquina, é executada uma micro-instrução.

em MARIN

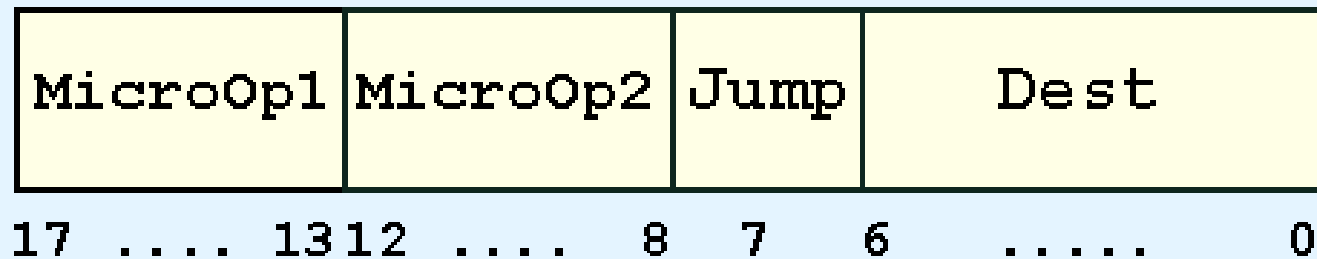


3.10.1 Descodificação de Instruções em MARIE

- Anteriormente forneceu-se a representação dos passos em RTL para todas as instruções do MARIE
- As micro-operações correspondentes a cada passo em RTL definem a operação da unidade de controlo
- Cada micro-operação consiste num padrão de sinais distinto que é interpretado pela unidade de controlo e resulta na execução da instrução MARIE original

3.10.1 Descodificação de Instruções em MARIE

- Se o MARIE fosse micro-programado, o formato das micro-instruções poderia ser assim:



- **MicroOp1** e **MicroOp2** : códigos binários de duas micro-operações (ou micro-operação + parâmetro)
- **Jump**: 1 bit, indicando que o campo **Dest** é um endereço válido e deve ser colocado no micro-sequenciador (i.e., a próxima micro-instrução a executar será a que reside neste endereço)

3.10.1 Descodificação de Instruções em MARIE

- Micro-operações e RTL correspondente:

MicroOp Code	Microoperation	MicroOp Code	Microoperation
00000	NOP	01100	$MBR \leftarrow M[MAR]$
00001	$AC \leftarrow 0$	01101	$OutREG \leftarrow AC$
00010	$AC \leftarrow AC - MBR$	01110	$PC \leftarrow IR[11-0]$
00011	$AC \leftarrow AC + MBR$	01111	$PC \leftarrow MBR$
00100	$AC \leftarrow InREG$	10000	$PC \leftarrow PC + 1$
00101	$IR \leftarrow M[MAR]$	10001	If $AC = 00$
00110	$M[MAR] \leftarrow MBR$	10010	If $AC > 0$
00111	$MAR \leftarrow IR[11-0]$	10011	If $AC < 0$
01000	$MAR \leftarrow MBR$	10100	If $IR[11-10] = 00$
01001	$MAR \leftarrow PC$	10101	If $IR[11-10] = 01$
01010	$MAR \leftarrow X$	10110	If $IR[11-10] = 10$
01011	$MBR \leftarrow AC$	10111	If $IR[15-12] =$ $MicroOp2[4-1]$

3.10.1 Descodificação de Instruções em MARIE

- Primeiras 9 linhas do microprograma (em RTL): primeiras 4 são o **fetch**; as restantes implementam o **decode** ...

Address	MicroOp 1	MicroOp 2	Jump	Dest
0000000	MAR $\leftarrow$ PC	NOF	0	0000000
0000001	IR $\leftarrow$ M[MAR]	NOF	0	0000000
0000010	PC $\leftarrow$ PC + 1	NOF	0	0000000
0000011	MAR $\leftarrow$ IR[11-0]	NOF	0	0000000
0000100	If IR[15-12] = MicroOp2[4-1]	00000	1	0100000
0000101	If IR[15-12] = MicroOp2[4-1]	00010	1	0100111
0000110	If IR[15-12] = MicroOp2[4-1]	00100	1	0101010
0000111	If IR[15-12] = MicroOp2[4-1]	00110	1	0101100
0001000	If IR[15-12] = MicroOp2[4-1]	01000	1	0101111
...	...	...	...	...

3.10.1 Descodificação de Instruções em MARIE

- **Add X** = {MAR $\leftarrow$ X; MBR $\leftarrow$ M[MAR]; AC $\leftarrow$ AC + MBR}
- Um **Add** é detetado pelo teste da **MicroOp1** no **Address** 0000111, pois **MicroOp2** [4-1]=0011 (opcode do **Add**)
- salta-se então (**Jump**=1) para o **Address** 0101100 (pois **Dest**=0101100), onde se vai “executar” o resto do **Add** (**MAR** $\leftarrow$ **X** já ocorreu na **MicroOp1** do **Address** 0000011)

Address	MicroOp1	MicroOp2	Jump	Dest
0101100	MBR $\leftarrow$ M[MAR]	NOP	0	0000000
0101100	AC $\leftarrow$ AC + MBR	NOP	1	0000000

- no fim da última micro-instrução do **Add** realiza-se um salto (**Jump**=1) para o início do microprograma (**Dest**=0000000), o que implica iniciar o fetch da próxima instrução MARIE ...

3.10.1 Descodificação de Instruções em MARIE

- É importante lembrar que uma unidade de controlo **microprogramada** opera como um **sistema em miniatura**
- Também as micro-instruções são sujeitas as fetch+decode+execute, tal como as instruções normais
- Este nível extra de interpretação torna a abordagem microprogramada mais lenta que o controlo hardwired
- Por outro lado, pode suportar instruções complicadas, e uma atualização ao microprograma permite acomodar novas instruções ou corrigir erros nas existentes

3.11 Arquiteturas Reais



- apesar de muito simples, a arquitetura MARIE inclui muita da funcionalidade das arquiteturas modernas
 - não é, todavia, uma representação exata dessas arquiteturas
 - sem mais registos, ou uma *stack*, a utilidade prática é limitada
- no mundo real, existem variadíssimas arquiteturas ...
- ... mas a arquitetura Intel (arquitetura CISC) e a MIPS (arquitetura RISC) são em si representativas
 - **CISC** = *complex instruction set computer*: grande número de instruções, de dimensão variável e de funcionalidade complexa;
 - **RISC** = *reduced instruction set computer*: pequeno número de instruções, de dimensão fixa e de funcionalidade simples; cada instrução executa rapidamente, mas os programas são maiores

3.11 Arquiteturas Reais



- a arquitetura Intel clássica, a 8086, nasceu em 1979 e trata-se de uma arquitetura CISC
- adotada pela IBM para o PC, lançado em 1981
- uma CPU 8086 operava em palavras de 16 bits e suportava endereços de 20 bits
- posteriormente, para baixar o preço, foi introduzido o 8088, que operava em palavras de 8 bits e usava também endereços de 20 bits

Qual era a capacidade máxima de memória do 8086 ?

3.11 Arquiteturas Reais



- o 8086 tinha **8** registos genéricos que permitiam acesso aos 16 bits (palavra completa) ou apenas a 8
 - **AX** (acumulador primário), **BX** (registo base, usado para endereçamento), **CX** (contador), **DX** (registo de dados)
 - AH, BH, CH, DH : parte mais significativa de AX, BX, CX, DX
 - AL, BL, CL, DL : parte menos significativa de AX, BX, CS, DX
 - **SP** (stack pointer), **BP** (base pointer), IP (instruction pointer)
 - **SI** (source index), **DI** (destination index): operações c/ strings
 - *status flags register*: overflow, parity, carry input, etc.
 - CS, DS, SS, ES (ver a seguir)

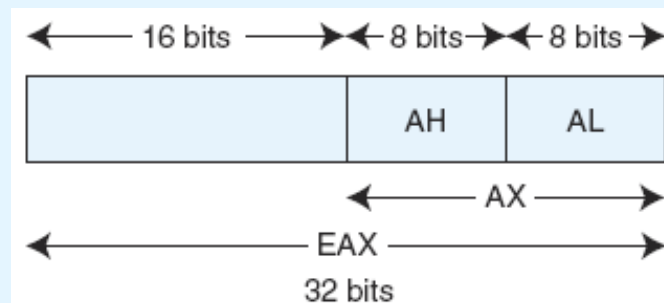
3.11 Arquiteturas Reais



- programas em assembly divididos em 4 segmentos
 - code segment (código do programa), referenciado por CS
 - data segment (dados do programa), referenciado por DS
 - stack segment (stack do programa), referenciado por SS
 - extra segment (p/ operações c/ strings), referenciado por ES
 - endereços do tipo <segmento, deslocamento no segmento>
- o 8086 não tinha unidade de vírgula flutuante ...
- em 1980, a Intel lançou o co-processador matemático 8087, que podia ser instalado com o processador 8086

3.11 Arquiteturas Reais

- em 1985, a Intel introduziu o 80386 (1º IA-32)
 - palavras de 32 bits, endereçava até 4Gbytes
 - também não tinha unidade de vírgula flutuante
- o 80486, introduzido em 1989, era um 80386 que tinha unidade de vírgula flutuante e memória *cache*
- o 80386 e 80486 eram compatíveis com o 8086 e 8088
 - o software desenvolvido para o 8086 apenas usa os 16 bits menos significativos dos registos de 32 bits do 80386 e do 80486



3.11 Arquiteturas Reais



- Pentium: registos de 32 bits, data bus de 64 bits, super-escalar (várias ALUs, + de 1 instrução por ciclo)
- Pentium Pro: *branch-prediction* (previsão de saltos)
- Pentium II: instruções MMX (vetoriais) p/ multimédia
- Pentium III: suporte adicional para gráficos 3D (FPU)
- Pentium 4: nova microarquitetura (*netburst*), *pipelining* (encadeamento de instruções) de > profundidade, *caches* multinível, > velocidades de relógio (> 3GHz)
- Core : > desempenho, < consumo, *multicore*, 64 bits
- a Intel tem vindo a adotar ideias do mundo RISC:
 - instruções CISC são traduzidas em instruções RISC

3.11 Arquiteturas Reais



- a família de CPUs MIPS (R3000, R4000, R5000, R8000 e R10000) é um caso de sucesso na sua classe (RISC)
- em 1986 foi anunciada a primeira CPU MIPS
 - o MIPS foi um dos primeiros processadores RISC
 - palavras de 32 bits, endereçava 4GB de memória
 - nº reduzido de instruções (55 contra >100 da 80806)
- atualidade: versões de 32 e 64 bits
 - MIPS32: 168 instruções, 32 registos genéricos
- cada nova versão procurou manter retro-compatibilidade

3.11 Arquiteturas Reais



- o desenho do MIPS é orientado ao desempenho:
 - arquitetura *load/store*: apenas as instruções *load* e *store* podem aceder à memória RAM
 - o elevado número de registos do MIPS mantém o tráfego do barramento em níveis mínimos
- os processadores MIPS têm sido usados em variados domínios: computadores, consolas de jogos, *routers*
- *SPIM*: simulador MIPS R2000/R3000 similar ao MARIE

Conclusões

- os principais componentes de um sistema de computação são a unidade de controlo, os registos, a memória, a ALU e o *datapath*
- um relógio mantém tudo sincronizado
- as unidades de controlo podem ser micro-programadas ou em hardware (*hardwired*)
- as unidades de controlo em hardware têm melhor desempenho, enquanto que as micro-programadas são mais adaptáveis

Conclusões

- os computadores executam programas obedecendo a ciclos *fetch-decode-execute*
- os computadores executam programas que se encontram expressos em linguagem máquina
- um *assembler* converte mnemónicas em linguagem *assembly* para linguagem máquina
- a arquitetura Intel é um exemplo de arquitetura CISC e a MIPS adotou a abordagem RISC